Assignment #2

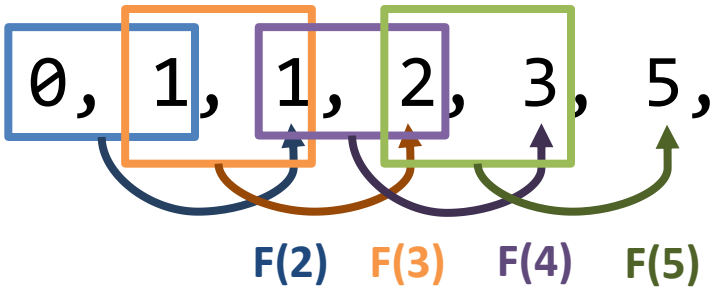
MIPS PROGRAMMING PROJECT

DEADLINE: 2026.11.09 (NOVEMBER 9TH)

Assignment #2

- Fibonacci Sequence

```
F(0) = 0
F(1) = 1
F(n) = F(n – 1) + F(n – 2), for n ≥ 2
```



- Goal

Write a Fibonacci program to print the sequence.

Factorial Example

(Core) MIPS code:

```
fact:
    addi $sp, $sp, -8       # adjust stack for 2 items
    sw   $ra, 4($sp)        # save return address
    sw   $a0, 0($sp)        # save argument
    slti $t0, $a0, 1        # test for n < 1
    beq  $t0, $zero, L1
    addi $v0, $zero, 1      # if so, result is 1
    addi $sp, $sp, 8        #   pop 2 items from stack
    jr   $ra                #   and return
L1: addi $a0, $a0, -1       # else decrement n
    jal  fact               # recursive call
R:  lw   $a0, 0($sp)        # restore original n
    lw   $ra, 4($sp)        #   and return address
    addi $sp, $sp, 8        # pop 2 items from stack
    mul  $v0, $a0, $v0      # multiply to get result
    jr   $ra                # and return
```

```
int fact (int n)
{
  if (n < 1) return 1;
  else return n * fact(n- 1);
}
```

Error #1

- No initialization of `$a0`

  ➔ `$a0` default is 0

Factorial Example

(Core) MIPS code:

```
fact:
    addi $sp, $sp, -8       # adjust stack for 2 items
    sw   $ra, 4($sp)        # save return address
    sw   $a0, 0($sp)        # save argument
    slti $t0, $a0, 1        # test for n < 1
    beq  $t0, $zero, L1
    addi $v0, $zero, 1      # if so, result is 1
    addi $sp, $sp, 8        #   pop 2 items from stack
    jr   $ra                #   and return
L1: addi $a0, $a0, -1       # else decrement n
    jal  fact               # recursive call
R:  lw   $a0, 0($sp)        # restore original n
    lw   $ra, 4($sp)        #   and return address
    addi $sp, $sp, 8        # pop 2 items from stack
    mul  $v0, $a0, $v0      # multiply to get result
    jr   $ra                # and return
```

```
int fact (int n)
{
  if (n < 1) return 1;
  else return n * fact(n- 1);
}
```

Error #2

- `jr` before `jal`
  - ➔ No `$ra` to jump back (default is 0x00000000)
  - ➔ Error ! invalid program counter value!

- Solution: Required certain function calls `fact` first

Factorial Example

```
main:
    addi $a0, $zero, 3
    jal  fact
    # Exit
$ra → addi $v0, $zero, 10
    syscall
fact:
    addi $sp, $sp, -8
    sw   $ra, 4($sp)
    sw   $a0, 0($sp)
…
```

Initialize $a0 = 3

Jump and link to fact
$ra is assigned.

Factorial Example

```
main:
    addi $a0, $zero, 3
    jal  fact
    # Exit
    addi $v0, $zero, 10
    syscall
fact:
    addi $sp, $sp, -8
    sw   $ra, 4($sp)
    sw   $a0, 0($sp)
…
```

Use for exit program;
Or your program will continue execution the next instruction

→ **$v0 = 10**

Basic Instructions | Extended (pseudo) Instructions | Directives | Syscalls | Exceptions | Macros

read integer	5		$v0 contains integer read
read float	6		$f0 contains float read
read double	7		$f0 contains double read
read string	8	$a0 = address of input buffer $a1 = maximum number of characters to read	*See note below table*
sbrk (allocate heap memory)	9	$a0 = number of bytes to allocate	$v0 contains address of allocated memory
exit (terminate execution)	10		
print character	11	$a0 = character to print	*See note below table*
read character	12		$v0 contains character read
		$a0 = address of	

Print Result

```
main:
    addi $a0, $zero, 3
    jal  fact
```

```
# Print result
add  $a0, $v0, $zero
addi $v0, $zero, 1
syscall
```

```
# Exit
addi $v0, $zero, 10
syscall
```

Use for exit program;
Or your program will continue execution the next instruction

Service	Code in $v0	Arguments
print integer	1	$a0 = integer to print
print float	2	$f12 = float to print
print double	3	$f12 = double to print
print string	4	$a0 = address of null-terminated string to print

Project Requirement

- **Input Integer Constrained**
  - $0 \leq n \leq 20$
  - Otherwise, display an error message and ask again.
  - You do not need to handle non-integer input.
- **Print F(0), F(1), …, F(n), then exit.**
  - E.g., n = 5, print 0, 1, 1, 2, 3, 5
- **Use a recursive (遞迴) procedure** named fib to calculate each term.
- Pass the argument n in `$a0` and return the result F(n) in `$v0`.
- **Include corresponding C/C++ pseudocode.**
- **Procedure and Stack**
  - Save the return address and any values needed after a recursive call.
  - Restore any $s registers you modify, and restore $sp before returning.
  - **Explain your stack frame layout in the report.**

Submission

File name:
StudentID_Assignment2_fib.asm
StudentID_Assignment2_fib.pdf
StudentID_Assignment2_fib.mp4

1. MIPS Code (.asm)

2. Project Report (.pdf)

➔ Include C/C++ pseudocode, MIPS code, stack frame layout, and test results

➔ Prepare in any format but transform into pdf

➔(Suggestion) Material to help you make the explanation in video.

3. Video (YouTube Link or .mp4)

➔ 3-10 minutes in English

➔ Assessment focuses on the technical understanding and clear explanation, **NOT** English grammar, pronunciation or the fluency.

Show and Explain in Your Video

1. **Demonstrate your program**
   Show the output for n = 0, 1, 5, 10, and 50.

2. **Explain your recursive logic**
   Use the corresponding C/C++ code to explain how your MIPS:
   - handles the base cases F(0) and F(1)
   - combines the results of F(n − 1) and F(n − 2).

3. **Demonstrate stack usage**
   - Use F(3) to show how a stack frame is created and released.
   - Explain how your code
     - preserves the first recursive result during the second call
     - uses the saved $ra to return to the correct instruction.

Your video will be partially evaluated by your peers.

Assessment

Criterion	Weight
Video explanation and MARS demonstration	40%
Program correctness, including stack management and calling convention	30%
Project report	20%
Peer Evaluation	10%

Factorial example: I

Service	Code in $v0	Arguments	
print integer	1	$a0 = integer to print	
print float	2	$f12 = float to print	
print double	3	$f12 = double to print	
print string	4	$a0 = address of null-terminated string to print	
read integer	5		$v0 contains integer read
read float	6		$f0 contains float read
read double	7		$f0 contains double read
read string	8	$a0 = address of input buffer $a1 = maximum number of	*See note below table*

```
main:
    addi $a0, $zero, 3
    jal  fact
```

Initialize the $a0 = 3

```
.data
prompt:
    .asciiz "Enter n (0-12): "
.text
main:
    la   $a0, prompt
    li   $v0, 4
    syscall
    li   $v0, 5
    syscall
    move $a0, $v0
```

Pseudo instruction:
la: Load address
li: Load integer

Print prompt

Read input *n*

Factorial example: Input Value

```
main:
    addi $a0, $zero, 3
    jal  fact
```

Initialize the $a0 = 3

```
.data
prompt:
    .asciiz "Enter n (0-12): "
.text
main:
    la   $a0, prompt
    li   $v0, 4
    syscall
    li   $v0, 5
    syscall
    move $a0, $v0
```

Put data in memory

The next following is program in .text

Print prompt

Read input *n*

Factorial example: Input Value

```
main:
    addi $a0, $zero, 3
    jal  fact
```

Initialize the $a0 = 3

Basic Instruction Version
➔Specify the detailed address

```
lui $a0, 0x1001
addi $v0, $zero, 4
syscall
```

.data starts from 0x10010000

```
.data
prompt:
    .asciiz "Enter n (0-12): "
```

Put data in memory

```
.text
```

The next following is program in .text

```
main:
    la   $a0, prompt
    li   $v0, 4
    syscall
```

Print prompt

```
    li   $v0, 5
    syscall
    move $a0, $v0
```

Read input *n*

Factorial example: Input Value

```
main:
    addi $a0, $zero, 3
    jal  fact
```

Initialize the $a0 = 3

Basic Instruction Version
➔Specify the detailed address

```
lui $a0, 0x1001
ori $a0, $a0, 0x0003
addi $v0, $zero, 4
syscall
```

Prompt storage starts from 0x10010003

Put data in memory

```
.data
message:
    .asciiz "Hi"
prompt:
    .asciiz "Enter n (0-12): "
```

End of string (null) takes 1 byte
To store 0x00

```
.text
main:
```

```
la   $a0, prompt
li   $v0, 4
syscall
```

Print prompt

```
li   $v0, 5
syscall
move $a0, $v0
```

Read input *n*